Istanbul Technical University

EHB326E - Assignment 2 Report

Interrupt Performance Effect on UART

Abdurrahim Yılmaz (820220519)

1 November 2026

1 Abstract

In this homework a UART program on the microBee (uBee, RV32IMC) prints the opening of "A Tale of Two Cities" (725 bytes, 230400 baud). At the same time an event generator that I wrote for the PL side makes periodic events. The same event is handled first with polling and then with an interrupt (PLIC). I measured CPU cycle loss, response latency, missed events and total application time with the uBee monitor counters, the CLINT timer and a latency counter in the event generator. I ran 19 experiments in Vivado behavioral simulation with event periods from 20000 to 50 clock cycles. The interrupt method was better when the period was 500 cycles or more (5.1 % CPU loss against 10.5 % at 1000 cycles) and its latency was always about 21 cycles. Polling used less CPU only for very frequent events (200 cycles or less), but it missed events at 100 cycles. The total time was 31.47 ms in all runs because the UART speed sets it. A non-blocking UART loop removed the CPU waiting time: with a blocking loop 95.5 % of the CPU time was lost.

2 Objective and Requirements

The objective is to find which event handling method (polling or interrupt) is better at which event rate, while the CPU also sends a text over the UART. The requirements of the topic are:

- a UART application that prints the opening of a book,
- a source that gives events (interrupts) at fixed intervals, handled first with polling and then with an interrupt,
- the same UART configuration, the same text and the same event sequence in both versions,
- measurement of CPU cycle loss, response latency and total performance at different event rates with the microBee system counters,
- not a higher baud rate, but fewer CPU cycles and less waiting time for the UART.

3 Method and System Design

Figure 1 shows the system. The uBee SoC IP and my module evt_gen are in one Vivado block design (hw2_system). evt_gen is an AXI4-Lite slave at 0x6000_0000 (4 KB). Its irq output goes to ext_irq_i[0], which is PLIC source 1. The testbench gives the settings of each experiment on GPIO IN, resets the system and decodes the UART output into the Tcl console and sim_log.txt. Figure 2 shows the block design in Vivado (the address editor screenshot is in the screenshots folder).

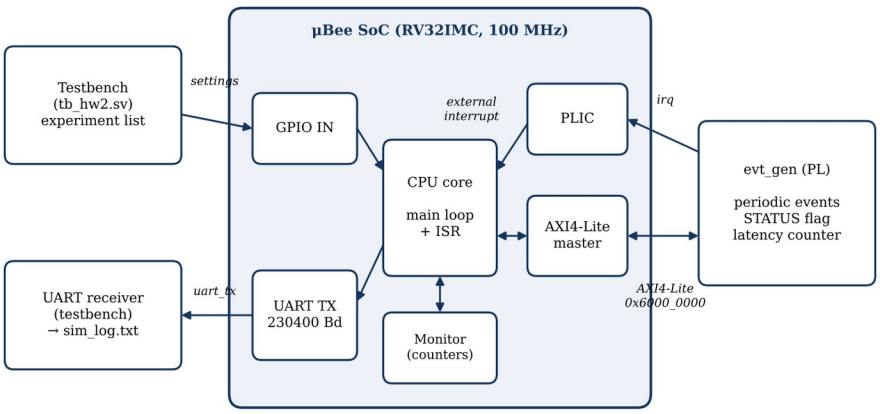


Figure 1. System overview.

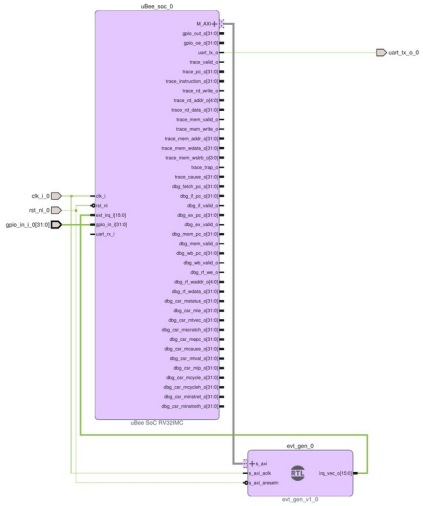


Figure 2. Vivado block design (hw2_system): uBee_soc_0 and evt_gen_0.

3.1 Event generator (evt_gen.v)

A counter counts down from PERIOD. At the end one event happens: EVT_CNT increases and PENDING becomes 1. The irq output is PENDING AND IRQ_EN, so the same flag is used for polling and for the interrupt. The CPU writes 1 to STATUS to acknowledge (ACK) the event. If a new event comes while PENDING is still 1, it is counted in MISS_CNT. While PENDING is 1, a counter counts every clock; at the ACK its value is added to LAT_SUM and LAT_MAX is updated. So the latency (event to ACK) is measured in hardware and the software timing does not change. Registers: CTRL, PERIOD, STATUS, EVT_CNT, ACK_CNT, MISS_CNT, LAT_SUM, LAT_MAX.

3.2 Software (hw2.s)

One assembly program is used for all experiments, so the UART code, the text and the event task are the same. It was assembled with the course rv32 tool (rv32 asm with ubee.toml). The program reads the mode and the period from GPIO IN and sets the UART divider to 434 (100 MHz / 434 = 230400 baud). In interrupt mode it also sets mtvec, the PLIC (priority, level mode, threshold, enable), mie.MEIE and mstatus.MIE. The main loop does three things: (1) in polling mode it reads STATUS and does the task if PENDING is 1, (2) if the TX FIFO is not full it writes the next character, but it never waits for the UART, (3) it runs a small calculation loop and increases a work counter. The task is the same in both modes: ACK, read CLINT mtime, save it as a time stamp, increase event_count. The normal ISR does not save registers, because in my uBee_soc the parameter SHADOW_BANK_ENABLE is 1 (IP default), so a trap uses a second register bank. For comparison I also wrote a slow ISR that saves 6 registers on a stack like a normal RISC-V ISR, and a blocking UART version that waits while the FIFO is full.

3.3 Measurement method

The window starts when the program clears and enables the uBee monitor and starts evt_gen, and it ends when the last character has left the UART. Monitor counters: CYCLES (total time), RETIRED (instructions), IRQ_TAKEN (interrupts) and AXI_WAIT (wait for the AXI bus). The monitor has no counter for interrupt latency (IRQ_TAKEN only counts the accepted interrupts), so the latency is counted in evt_gen (3.1) and checked on the waveforms. One character takes 10 x 434 = 4340 cycles, so the total time is fixed by the baud rate. For this reason the CPU loss is measured with the work counter. Experiment 1 (interrupt mode, no events) is the reference, W_ref = 33,015:

- CPU loss = (W_ref - W) / W_ref x 100 %
- average latency = LAT_SUM / ACK_CNT, missed events = MISS_CNT / EVT_CNT x 100 %
- time stamp check (CLINT mtime): last time stamp / event_count = mean time between handled events

The monitor cost was measured with an empty measurement (3 cycles), so no correction was needed. The test plan has 19 experiments: polling and interrupt at 7 periods (20000, 5000, 1000, 500, 200, 100, 50 cycles, that is 5 kHz to 2 MHz), both modes without events, the slow ISR at 1000 and 200 cycles, and the blocking UART without events.

4 Simulation and Findings

The behavioral simulation was run in Vivado 2026.1 (XSim) with run all. It took about 37 minutes. All 19 experiments finished and printed the full text, the RESULT line and END (screenshots/03_tcl_console_end.png). Table 1 and Figure 3 were made from sim_log.txt with my Python script analyze.py.

Table 1. CPU loss, latency (avg / max, cycles) and missed events

Period (cycles)	Rate	Poll loss %	IRQ loss %	Poll latency	IRQ latency	Poll missed %	IRQ missed %
no events	-	7.8	0.0	-	-	-	-
20,000	5 kHz	7.9	0.2	62.5 / 114	21.3 / 23	0.0	0.0
5,000	20 kHz	8.3	1.0	64.5 / 128	21.2 / 23	0.0	0.0
1,000	100 kHz	10.5	5.1	64.1 / 128	21.3 / 23	0.0	0.0
500	200 kHz	13.3	10.1	64.2 / 129	21.3 / 23	0.0	0.0
200	500 kHz	20.9	25.4	59.9 / 123	21.5 / 23	0.0	0.0
100	1 MHz	26.9	50.7	79.3 / 142	21.4 / 24	23.3	0.0
50	2 MHz	26.9	96.8	104.3 / 142	19.3 / 24	61.7	0.0

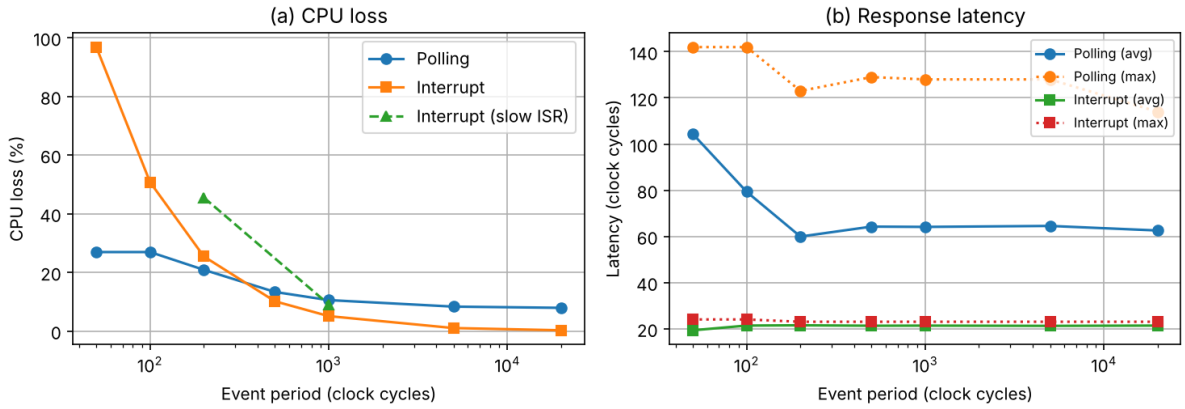


Figure 3. CPU loss and response latency against the event period (log scale).



Figure 4. Waveforms at a period of 1000 cycles, four events each. Top: polling (experiment 6). Bottom: interrupt (experiment 7). lat_now keeps the latency of the last event.

Findings:

- Total time was 31.47 to 31.48 ms in all runs (expected 725 x 4340 = 3,146,500 cycles). The event method changes only the free CPU time, not the UART time.
- Polling costs 7.8 % even without events, because every loop reads STATUS over AXI (AXI_WAIT = 91,356 cycles). The interrupt costs nothing when there is no event.

- Lost cycles per event (loss x 95.3 cycles per loop / events): about 51 for the interrupt, 91 for the slow ISR and 27 for polling. The interrupt needs a pipeline drain, the trap, PLIC claim/complete and mret.
- So polling loss = 7.8 % + 27 cycles per event and interrupt loss = 51 cycles per event. They are equal at about 300 cycles (about 330 kHz). This agrees with Table 1: at 500 cycles the interrupt is better, at 200 cycles polling is better.
- The interrupt latency was 19 to 22 cycles on average and at most 24. The polling latency depends on where the loop is, so it changes from event to event (Figure 4a) and its maximum was up to 142 cycles.
- Polling can answer one event per loop. The mtime check shows one handled event every 130 cycles, so at 100 and 50 cycles 23 % and 62 % of the events were missed. The interrupt missed nothing, but at 50 cycles the CPU loss was 97 %.
- Saving registers in the slow ISR made the loss 9.1 % instead of 5.1 % (1000 cycles) and the latency 41 instead of 21 cycles. The shadow register bank of uBee is a real gain.
- With the blocking UART loop the work counter was only 1,501, so 95.5 % of the CPU time was spent waiting for the UART. The non-blocking loop gives this time back to the program without changing the baud rate.

5 Verification and Evaluation

Table 2. Test and observation records

Test	Input / condition	Expected result	Observed result
1	All 19 experiments	Full text, then RESULT and END	Same 725-byte text in all runs, no time-out
2	Events on	EVT_CNT = ACK_CNT + MISS_CNT (+1 if one is pending at the end)	True in all runs
3	Events on	Software event_count = ACK_CNT	True in all runs
4	Interrupt mode	One interrupt per event: IRQ_TAKEN = ACK_CNT	Equal (difference at most 3 at 50 cycles, after the monitor freeze)
5	CLINT time stamps	last time / event_count = period (no misses)	1000.1 cycles at P = 1000, 200.0 at P = 200
6	UART 230400 baud	CYCLES = 3,146,500	3,146,554 to 3,148,161
7	Waveforms	PENDING clears after ACK; trap only in interrupt mode; lat_now = cycles from event to ACK	As expected (Figure 4)

The design works correctly: all experiments finished, the counters agree with each other and the UART output is the same in every run. My decision for a 100 MHz uBee: use the interrupt when the event period is 500 cycles or more (200 kHz or less), because it has less CPU loss and a short, stable latency. Between 200 and 500 cycles both are close; the interrupt is better if latency is important. Use polling only for 100 to 200 cycles and only if a long latency is acceptable. Below 100 cycles neither method is good: polling loses events and the interrupt uses almost all of the CPU time. Limitations: there is only one event source and the event task is very small.

6 Conclusion

I added a periodic event generator to the uBee system and handled its events with polling and with an interrupt while the CPU printed a text over the UART. The uBee monitor counters, the CLINT timer and a hardware latency counter were used in 19 Vivado simulations. The UART time did not change, but the free CPU time changed a lot. The interrupt is better for events with a period of 500 cycles or more; polling is cheaper only for very frequent events and then it has long latency and lost events. The shadow register bank makes the interrupt about 44 % cheaper, and the non-blocking UART loop removes the CPU waiting time. The objectives of the topic are met.

References

[1] ITU GSTL, microBee Technical Reference (microBee-TR-001), 2026.
[2] S. E. Önsay, F. A. Güler, A. E. Kıyak, M. E. Yalçın, EHB326E RISC-V MCU Book (microBee-DS-001), 2026.
[3] GSTL-ITU, uBee repository and rv32 emulator, https://github.com/GSTL-ITU/uBee, accessed: 01.11.2026.
[4] RISC-V International, The RISC-V Instruction Set Manual, Volume II: Privileged Architecture.
[5] AMD, Vivado Design Suite User Guide: Logic Simulation (UG900).